

Consegna 5a - OpenMP

L'obiettivo del laboratorio è la parallelizzazione, tramite OpenMP, di un programma in linguaggio C dedicato al calcolo approssimato di π (Pi Greco) mediante il metodo dei rettangoli applicato a un integrale definito.

Successivamente, viene studiato lo scaling prestazionale al variare del carico computazionale (numero di intervalli N) e del parallelismo (numero di thread OpenMP), mettendo a confronto un'architettura consumer (PC laptop) e un'architettura enterprise (cluster HPC).

Configurazione Hardware degli Ambienti di Calcolo

1. PC Locale:

- **CPU:** Intel Core i7-1165G7.
- **Core Fisici:** 4 core dedicati.
- **Core Logici (Hyper-Threading):** 8 thread attivi complessivi.
- **Ambiente operativo:** Linux Ubuntu su sottosistema WSL2.

2. Cluster HPC (`wn20` - partizione `cpu_guest`):

- **CPU:** AMD EPYC 7282.
- **Core Fisici online:** 32 core fisici allocati per il job.
- **Hyper-Threading:** Disattivato (rapporto standard 1 thread hardware per core fisico) per garantire la massima determinazione delle performance.

Implementazione e Parallelizzazione del Codice (`omp_cpi.c`)

La parallelizzazione è stata ottenuta inserendo la direttiva `#pragma omp parallel for` opportunamente configurata per evitare *data race* e garantire la riduzione matematica:

```
double f2 (long int n)
{
    long int i;
    double x, sum=0.0;

    #pragma omp parallel for private(x) shared(h) reduction(+:sum) schedule(static)
    for (i = 1; i <= n; i++)
    {
        x = h * ((double)i - 0.5);
        sum += (1.0 / (1.0 + x*x));
    }
    return sum;
}
```

- `parallel for` : divide automaticamente le iterazioni del ciclo `for` tra i thread OpenMP disponibili.
- `private(x)` : rende la variabile d'appoggio `x` locale e indipendente per ogni thread, evitando che un thread sovrascriva il valore calcolato da un altro (*data race*).

- `shared(h)` : condivide la variabile `h` (ampiezza dell'intervallo) tra tutti i thread in modalità di sola lettura, risparmiando memoria.
- `reduction(+:sum)` : ogni thread calcola una somma parziale nella propria copia locale di `sum`. Al termine del ciclo, OpenMP combina (somma) in modo atomico i risultati parziali nella variabile globale `sum`.
- `schedule(static)` : distribuisce le iterazioni ai thread in blocchi di uguali dimensioni all'avvio del ciclo, riducendo al minimo l'overhead di scheduling dinamico.

Analisi dei Risultati e degli Output di Scaling

Il test di scaling è stato eseguito per tre livelli di granularità del problema ($N = 10^9, 2 \times 10^9, 4 \times 10^9$ intervalli) incrementando i thread OpenMP a 1, 2, 4, 8, 16 e 32.

Analisi Dati ed Evoluzione dei Grafici

Osservando i file `omp_cpi_scaling.csv` ed i grafici generati con Matplotlib, si evidenziano andamenti divergenti:

- **Sul Cluster (wn20):**

Prendendo il caso limite con $N = 4.000.000.000$:

- Con **1 Thread**: il tempo è di `5.09179` secondi.
- Con **2 Thread**: il tempo si dimezza a `2.55201` secondi (Speedup $S \approx 1.99$).
- Con **4 Thread**: il tempo scende a `1.28282` secondi (Speedup $S \approx 3.97$).
- L'andamento decrescente prosegue in modo pulito e lineare fino a **32 Thread** (`0.18354` secondi, con uno Speedup di ≈ 27.81).

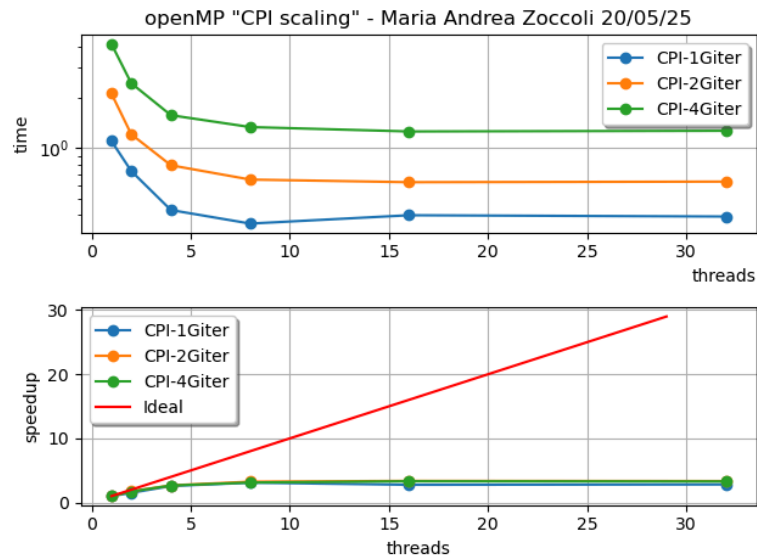
- **Sul PC Locale (andrea):**

Sullo stesso carico di lavoro ($N = 4.000.000.000$):

- Con **1 Thread**: il tempo iniziale è di `4.16322` secondi.
- Con **2 Thread**: si attesta a `2.43210` secondi (Speedup $S \approx 1.71$).
- Con **4 Thread**: scende a `1.56885` secondi (Speedup $S \approx 2.65$).
- A **8 Thread** la curva subisce un drastico appiattimento (`1.33353` s) ed oltre (16 e 32 thread) si nota una totale stagnazione o un leggero peggioramento delle performance dovuto all'overhead dovuto al context-switching della shell.

I grafici rispecchiano fedelmente i limiti e le potenzialità fisiche dei rispettivi processori:

1. **PC Locale**: il calcolo di Pi Greco tramite metodo dei rettangoli è un problema a forte densità computazionale basato quasi interamente su operazioni in **virgola mobile** (`double`). La CPU Intel del PC possiede 4 core fisici. Quando si impostano 8 thread OpenMP, entrano in gioco i core logici della tecnologia Hyper-Threading. Tuttavia, l'Hyper-Threading duplica solo i registri di stato e di controllo, ma **condivide le unità di calcolo matematico (ALU e FPU)** interne al core fisico. Trovandosi le FPU già sature dal calcolo parallelo intensivo dei primi 4 thread, gli altri 4 thread logici rimangono in coda aspettando la liberazione dell'hardware. Questo spiega la saturazione e l'appiattimento visibile nel grafico `grafico_scaling_locale.png` non appena si superano i 4 thread.



2. **Cluster HPC:** al contrario, sul nodo del cluster la partizione offre core reali indipendenti senza meccanismi di SMT/Hyper-Threading attivi. Ogni thread OpenMP viene mappato direttamente su una risorsa hardware dedicata ed esclusiva. Poiché i thread lavorano interamente su registri e cache locali, non si generano colli di bottiglia sul bus di memoria o sulle unità funzionali, consentendo alla curva del grafico `grafico_scaling_cluster.png` di decrescere in modo ottimale fino a 32 thread stabili.

